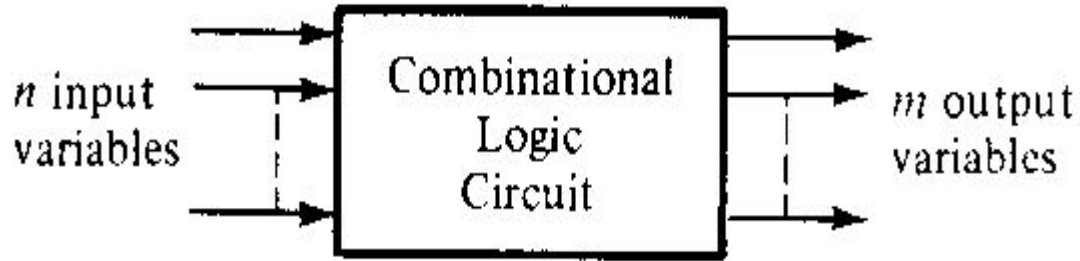


Combinational Circuits



- O/P depends only on present combination of I/Ps
- Consists of input variables, logic gates and output variables not memory elements

Design Procedure

1. State the problem
2. Determine the number of input variables and required output variables and assign them letter symbols
3. Derive truth table defining relationship between inputs and outputs
4. Simplify boolean function for each output
5. Draw the logic diagram

Half Adder

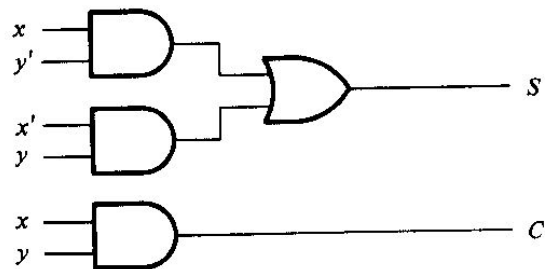
x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = x'y + xy'$$

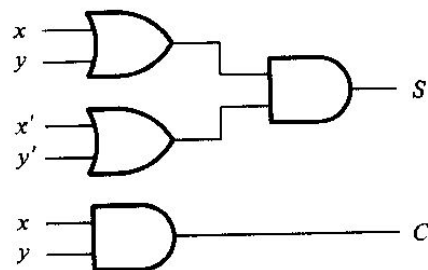
$$C = xy$$

SOP form



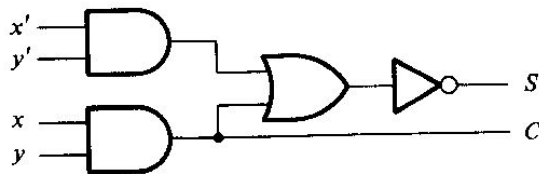
$$(a) \begin{aligned} S &= xy' + x'y \\ C &= xy \end{aligned}$$

POS form

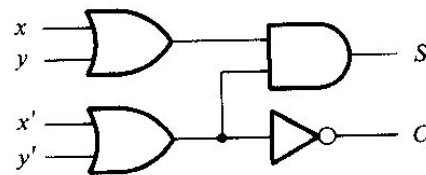


$$(b) \begin{aligned} S &= (x + y)(x' + y') \\ C &= xy \end{aligned}$$

$$\begin{aligned} S &= x'y + xy' \\ S' &= xy + x'y' \\ S &= (C + x'y')' \end{aligned}$$



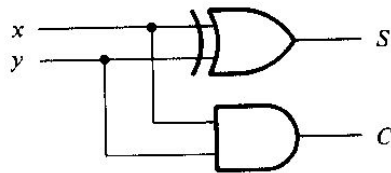
$$(c) \begin{aligned} S &= (C + x'y')' \\ C &= xy \end{aligned}$$



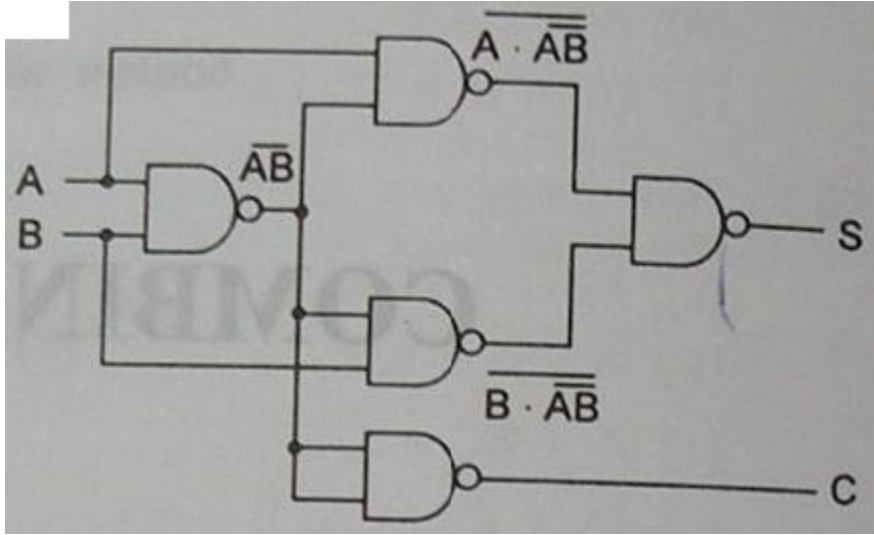
$$(d) \begin{aligned} S &= (x + y)(x' + y') \\ C &= (x' + y')' \end{aligned}$$

$$\begin{aligned} S &= (x+y)(x'+y') \\ C &= xy = (x'+y')' \\ S &= (x+y)C' \end{aligned}$$

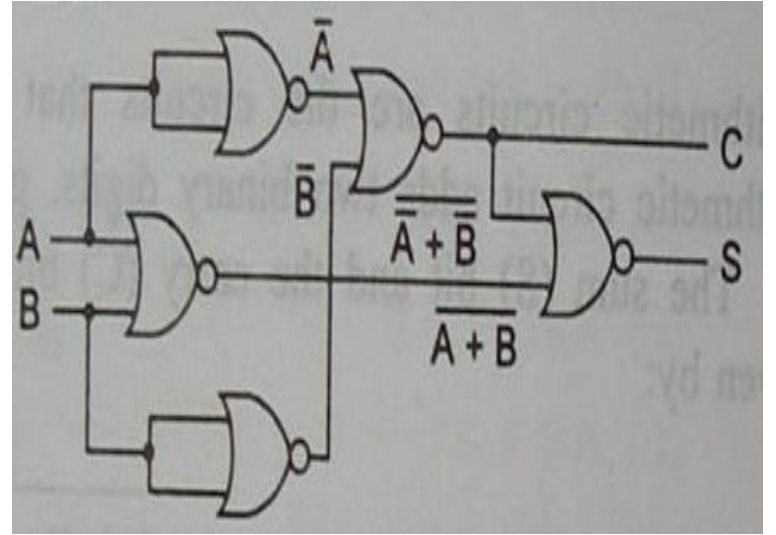
Using XOR



$$(e) \begin{aligned} S &= x \oplus y \\ C &= xy \end{aligned}$$



NAND Implementation



NOR Implementation

Full Adder

x	y	z	C	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

		yz		y	
		00	01	11	10
x	0		1		1
x	1	1		1	

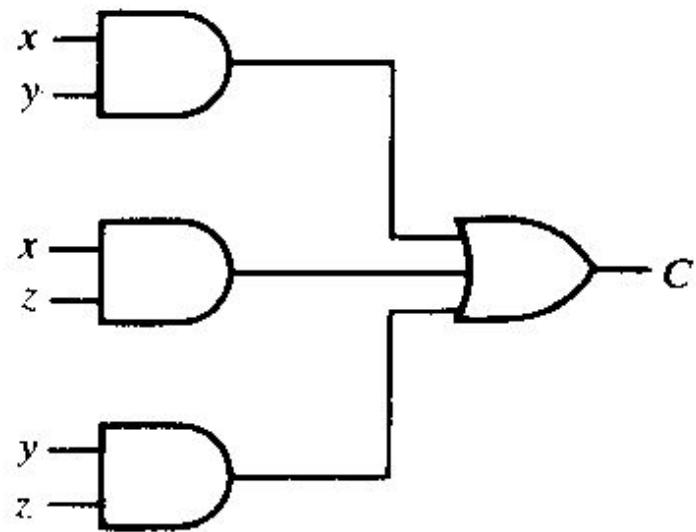
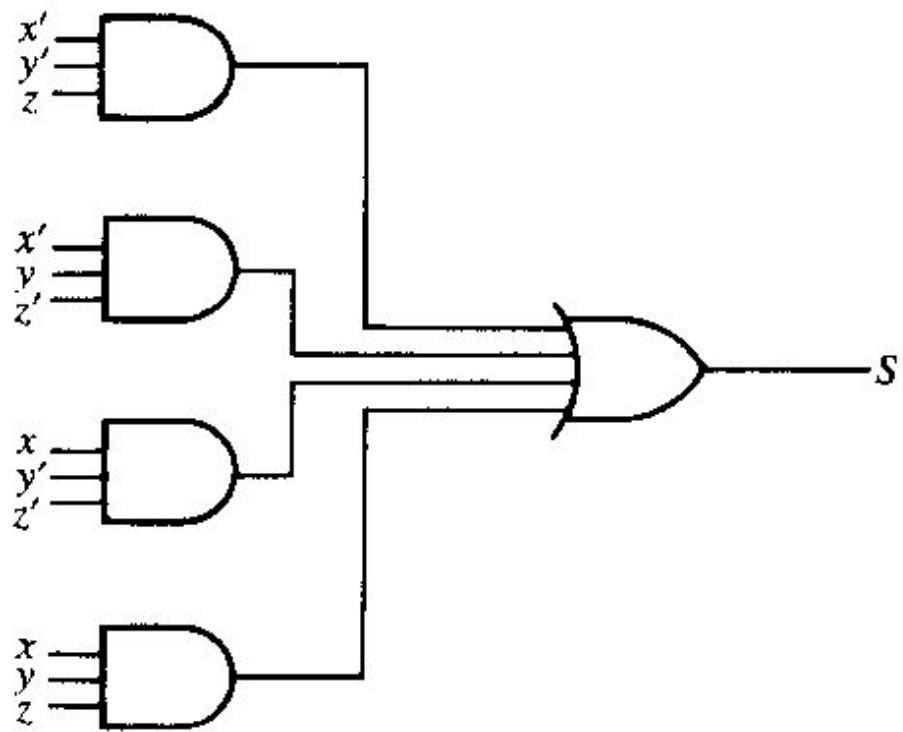
$$S = x'y'z + x'yz' + xy'z' + xyz$$

		yz		y	
		00	01	11	10
x	0			1	
x	1		1	1	1

$$C = xy + xz + yz$$

$$S = x'y'z + x'yz' + xy'z' + xyz$$

$$C = xy + xz + yz$$



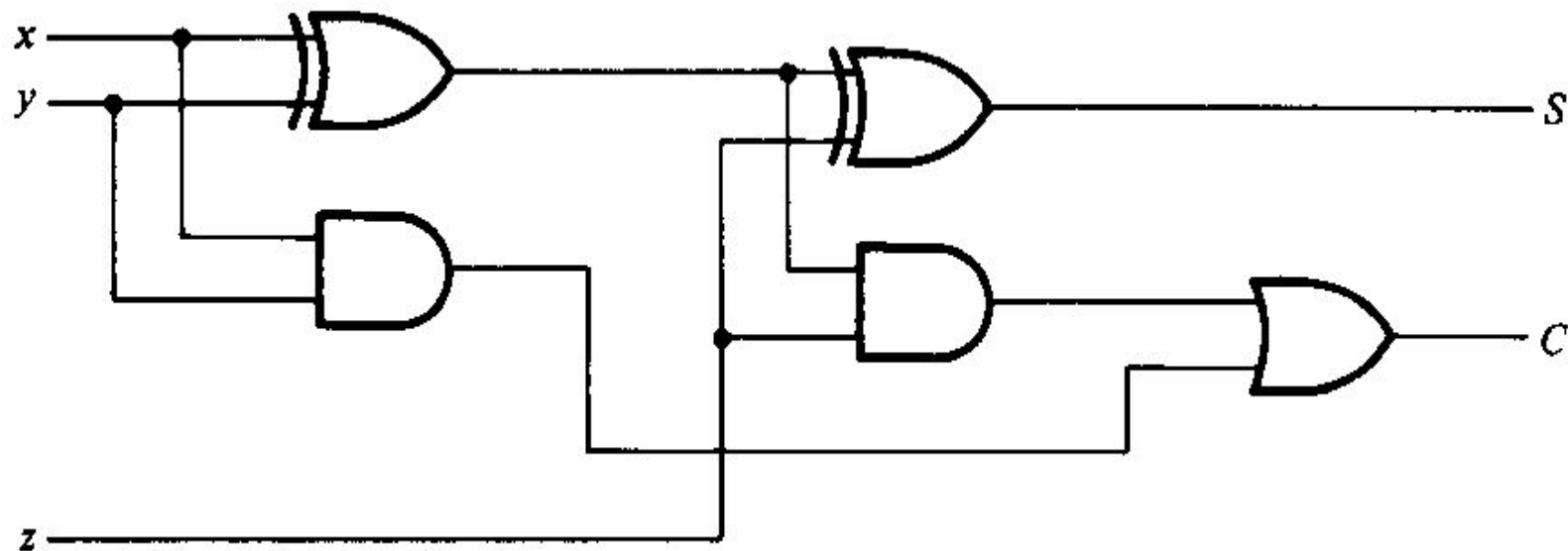
$$S = z \oplus (x \oplus y)$$

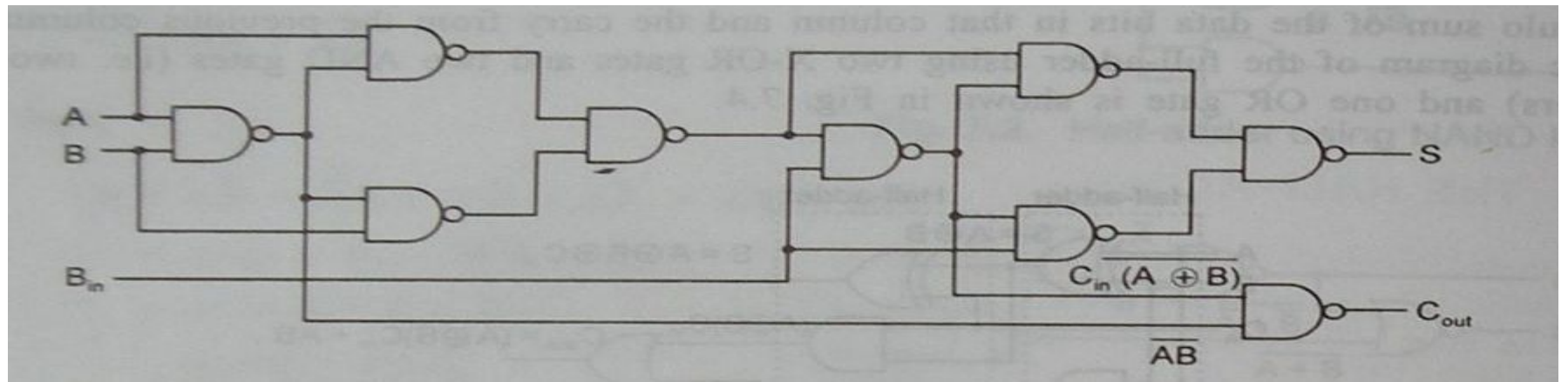
$$= z'(xy' + x'y) + z(xy' + x'y)'$$

$$= z'(xy' + x'y) + z(xy + x'y')$$

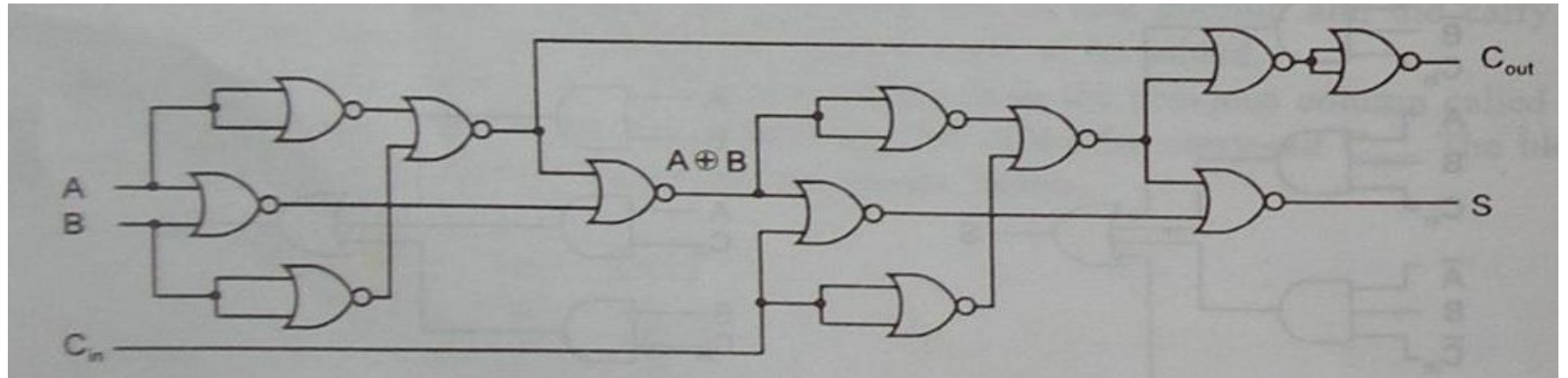
$$= xy'z' + x'yz' + xyz + x'y'z$$

$$C = z(xy' + x'y) + xy = xy'z + x'yz + xy$$





NAND Implementation



NOR Implementation

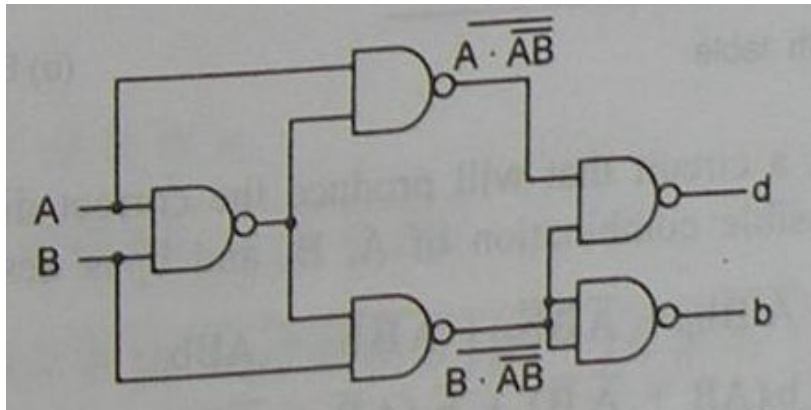
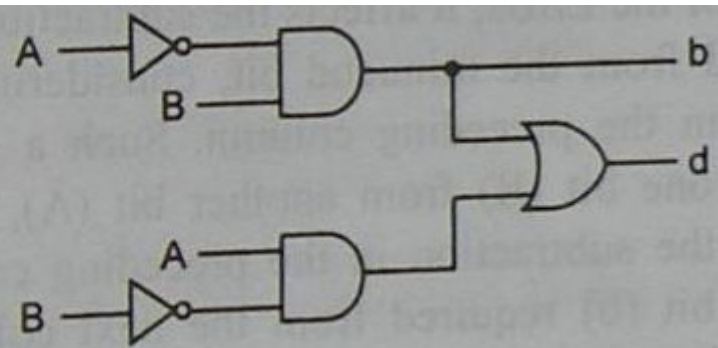
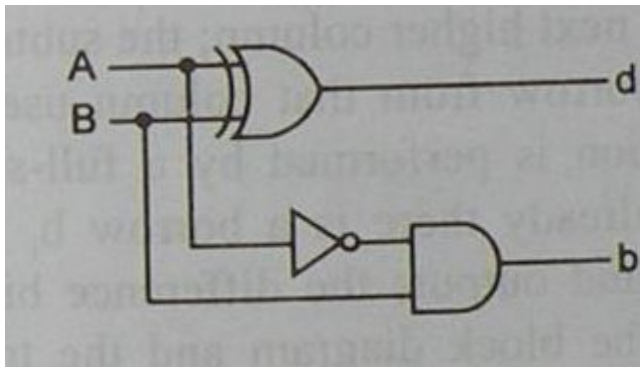
Half Subtractor

x	y	B	D
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

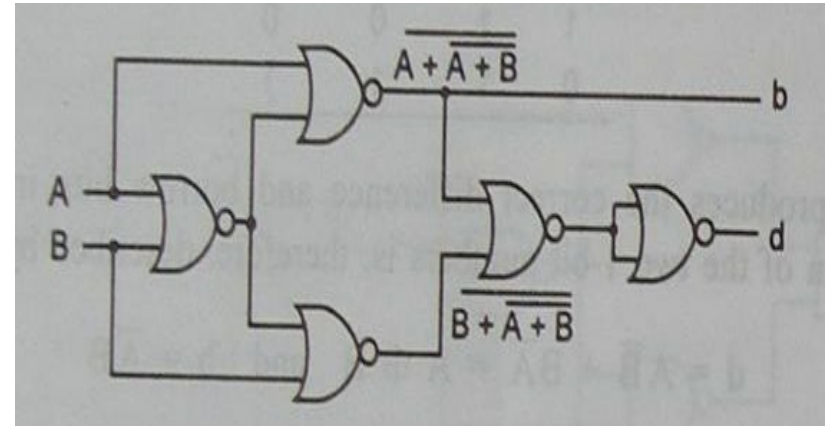
x	y	B	D
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

$$D = x'y + xy'$$

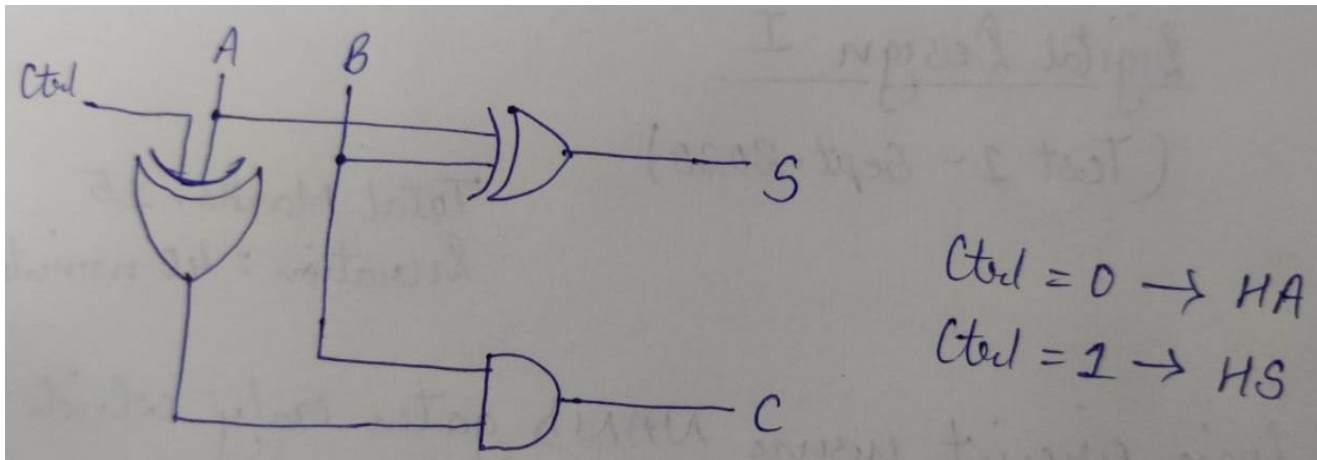
$$B = x'y$$



NAND Implementation



NOR Implementation



XOR

Control = 0 : Half Adder

Control = 1 : Half Subtractor

XNOR

Control = 1 : Half Adder

Control = 0 : Half Subtractor

Full Subtractor

x	y	z	B	D
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

x	y	z	B	D
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

		yz		y	
		00	01	11	10
x	0		1		1
x	1	1		1	
		z			

$$D = x'y'z + x'yz + xy'z' + xyz$$

$$D = x'y'z + x'yz' + xy'z' + xyz$$

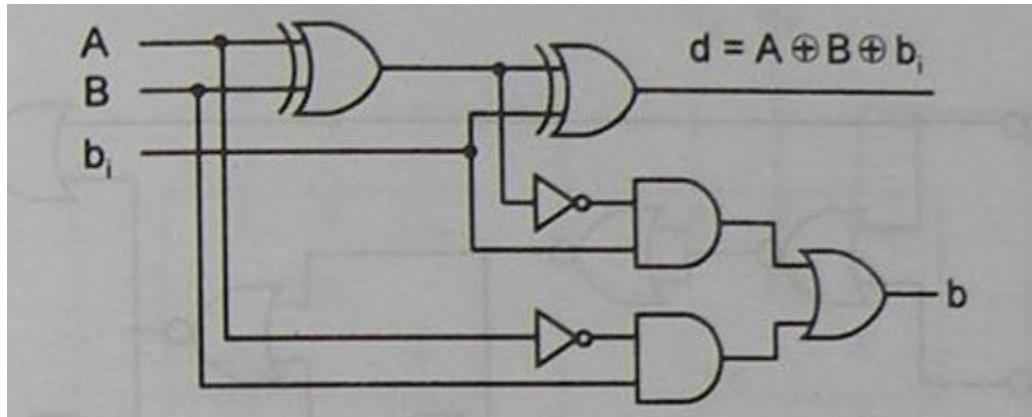
$$B = x'y + x'z + yz$$

$$D = x \oplus y \oplus z$$

$$B = (x \oplus y)'z + x'y$$

		yz		y	
		00	01	11	10
x	0		1	1	1
x	1			1	
		z			

$$B = x'y + x'z + yz$$



$$D = x'y'z + x'yz' + xy'z' + xyz$$

$$B = x'y + x'z + yz$$

$$D = x \oplus y \oplus z$$

$$B = (x \oplus y)'z + x'y$$

XOR

Control = 0 : Full Adder

Control = 1 : Full Subtractor

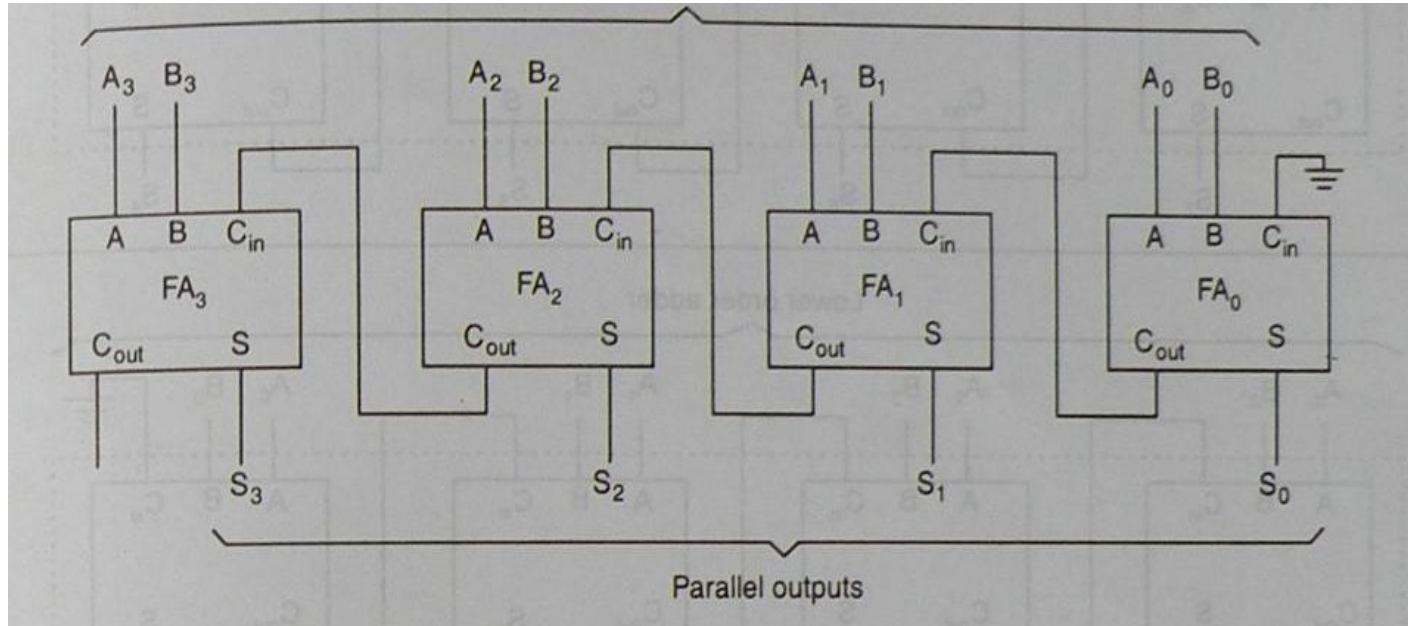
XNOR

Control = 1 : Full Adder

Control = 0 : Full Subtractor

Parallel Adder (or) Ripple Carry Adder

- Used to add group of bits



n-bit addition:

- n FA
 - (n-1) FA and 1 HA
 - (2n-1) HA and (n-1) OR gates
-
- The time required for to perform a valid addition depends on the number of bits to be added.
Greater the number of bits, the greater the time required

$$1 \text{ addition delay} = (n-1) t_{pd\text{carry}} + (t_{pd\text{carry}} \text{ or } t_{pd\text{sum}})_{\text{max}}$$

$$\text{Rate of addition} = 1 / (n * t_{pd\text{FA}}) \text{ addition/sec}$$